

UAV-Enabled ISAC for Livestock Monitoring

A from-zero guide to the genuine Deep Reinforcement Learning implementation

What the system does • what every important word means • what was changed • how it was tested • how to run it • how to report it honestly

The one-sentence version

We turned a simulator that previously tested one fixed UAV configuration per run into an interactive learning environment: every 10 simulated seconds, a Double-DQN agent observes the current mission, chooses one UAV control action, and sees the consequence in the next 10-second window.

Prepared from the verified code and artifacts on 18 August 2026

How to use this guide

This document is intentionally written for a reader who has never studied reinforcement learning, ns-3, ISAC, or UAV control. Read Chapters 1–5 for the story and vocabulary. Chapters 6–13 explain the technical design slowly. Chapters 14–18 show testing, results, commands, paper-writing rules, and remaining work.

Important scientific warning

The completed three-episode experiment is a smoke test. It proves that the software loop runs; it does not prove that DRL is better. A full multi-seed campaign is still required before adding a DRL improvement claim to the conference paper.

Contents

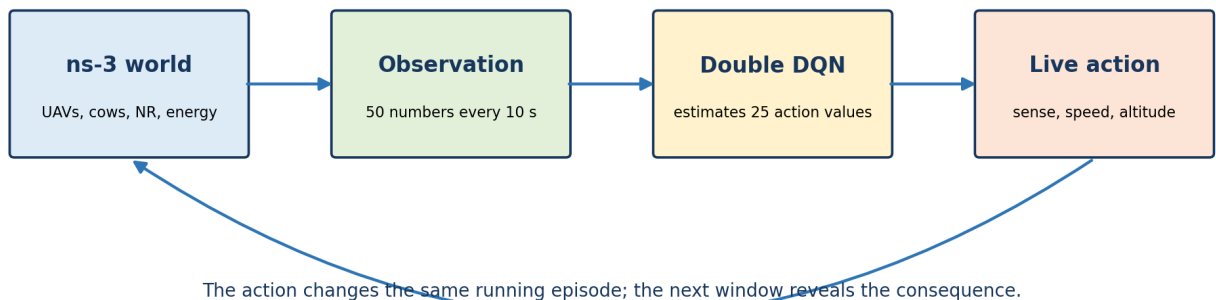
1. The problem in everyday language

Imagine a large farm with 80 cows and four flying robots. A farmer wants to know where the cows are. The drones must fly around, detect cows, estimate their positions, and send reports through a 5G base station. They cannot waste battery, and the radio link is not perfect.

The difficult part is that a good decision changes over time. When few cows are detected, a drone may need to sense more frequently or move differently. When battery use is high, it may need to reduce effort. A fixed setting selected before take-off cannot react to what happens during the mission. DRL is introduced to learn these repeated decisions.

- Sensing question: Are the cows detected, and how accurate are their estimated locations?
- Communication question: Do reports arrive quickly, without loss, and at useful throughput?
- Mobility question: Where should each drone fly, how fast, and at what altitude?
- Energy question: Can the mission be completed within each drone's battery budget?
- Learning question: Which action now is likely to produce good total performance later?

2. The complete system at a glance



One loop works as follows: ns-3 simulates ten seconds; it summarizes the new sensing, network, energy, position, and herd information; Python receives 50 numbers; Double DQN chooses one of 25 actions; ns-3 changes the live UAV object; then another ten seconds are simulated. This repeats until time expires or a battery is exhausted.

Why the word live matters

Changing only a Python variable or a C++ command-line value after a simulation starts does not control the current drone. The implemented setters modify the mobility and sensing objects that are already running inside ns-3.

3. Essential vocabulary—from the beginning

Keyword	Plain expansion	Meaning here
UAV	Unmanned Aerial Vehicle	A drone. Four are simulated with different roles.
ISAC	Integrated Sensing and Communication	The UAV senses cows and communicates reports in one system.
5G NR	Fifth-generation New Radio	The cellular radio technology used for UAV-to-network traffic.
gNB	Next-generation Node B	The 5G base station.
UE	User Equipment	A device connected to the gNB; each UAV acts as a UE.
ns-3	Network Simulator 3	A discrete-event simulator that models time, packets, nodes, mobility, and radio links.
KPI	Key Performance Indicator	A measured quantity such as detection probability, delay, or energy.
RL	Reinforcement Learning	Learning actions by interacting with an environment and receiving rewards.
DRL	Deep Reinforcement Learning	RL that uses a neural network to represent a policy or value function.
Agent	Decision maker	Here, the Double-DQN program that chooses UAV actions.
Environment	The world the agent controls	Here, the running ns-3 livestock/UAV simulation.
State/observation	What the agent sees	A 50-number summary at a decision time.
Action	What the agent does	One of 25 sensing, speed, altitude, or no-op choices.
Reward	Immediate score	A number rewarding sensing/network quality and

Keyword	Plain expansion	Meaning here
		penalising energy.
Policy	Decision rule	A mapping from an observed state to an action.
Episode	One complete mission	One ns-3 run from reset to battery termination or time limit.
Step	One decision cycle	One action followed by one 10-second simulator window.
Transition	One learning example	State, action, reward, next state, and done flag.
DQN	Deep Q-Network	A neural network that estimates the long-term value of every discrete action.
Double DQN	Two-network target rule	Online network selects the next action; target network evaluates it.
Q-value	Expected discounted return	How good an action is now, including likely future rewards.
Bellman target	Learning target	Immediate reward plus discounted value of the next state.
Replay memory	Transition storage	A buffer that mixes old experiences into training batches.
Target network	Slow reference network	A copied network that stabilises the Bellman targets.
Epsilon	Exploration probability	Chance of taking a random action instead of the current greedy choice.
Hyperparameter	A chosen learning setting	For example learning rate, batch size, or discount factor.
Seed	Random-number starting point	Makes stochastic experiments repeatable and enables matched comparisons.
Baseline	Reference method	Static, random, rule-based, or RF control used for

Keyword	Plain expansion	Meaning here
		comparison.
RF	Random Forest	A supervised regression benchmark; it is not RL in this project.
O-RAN/xApp	Programmable RAN controller	A possible deployment route for a trained, safety-checked policy.
Smoke test	Small execution check	Confirms the pipeline runs; it is not a performance experiment.

4. What was wrong with the earlier 'RL' claim?

The released file called `offline_rl_analysis.py` trained a Random Forest regressor. Each row described a complete fixed simulator configuration and its final J score. The model predicted which new fixed configurations might score well. That is supervised surrogate modelling plus candidate search.

Question	Released RF method	Implemented DRL method
Does it act during an episode?	No	Yes, every 10 seconds
Does it use ordered state transitions?	No	Yes
Does an action affect the next state?	No sequential link	Yes, through live ns-3 control
Bellman equation?	No	Yes
Replay memory?	No	Yes
Target neural network?	No	Yes
Correct name	RF surrogate/random-search benchmark	Double-DQN controller

This does not make the RF method useless

The RF method remains a valuable baseline. It answers a different question: which fixed configuration looks promising? The DRL agent answers: given the mission now, what should I change for the next window?

5. Reinforcement learning with a simple analogy

Think of teaching someone to manage four sprinklers on a farm. Every ten minutes they see soil moisture, weather, and water remaining. They can adjust one sprinkler. Healthy crops give positive points; wasted water gives negative points. After many seasons, they learn which adjustments tend to produce good long-term outcomes. Replace sprinklers with UAV controls, soil information with KPIs, and seasons with simulated missions: that is the basic RL idea.

- The agent is not given a perfect answer for every state.
- It tries actions, sees consequences, and stores experiences.
- It must explore enough to discover alternatives.
- It must also exploit what it has already learned.
- A reward must represent the actual scientific objective; otherwise the agent learns the wrong behaviour very efficiently.

6. What exactly is one episode and one step?

An episode is one complete ns-3 mission. At reset, cow motion, radio randomness, and UAV motion are initialised using a recorded seed. With the standard 300-second mission and a 10-second control interval, there are approximately 29 actions after the first observation at 10 seconds. The final transition arrives at 300 seconds.

Simulated time	What happens
0–10 s	ns-3 runs without an RL action to produce the first meaningful window.
10 s	Python receives state 0 and sends action 0.
10–20 s	The selected action affects live sensing or mobility.
20 s	Python receives reward and state 1, then sends the next action.
...	The locked state/action sequence continues.
300 s	The time-limit transition is marked truncated; the episode ends.
Battery exhausted	The transition is marked terminated; the episode ends early.

7. The 50-number observation

A neural network cannot directly read ‘the patrol UAV is high and the link is slow.’ It receives numbers. The observation deliberately combines what the agent needs to understand quality, physical state, active controls, and remaining mission time.

Indices	Count	Contents	Why included
0–4	5	Detection, RMSE, throughput, delay, loss	Quality of the most recent window
5–8	4	Remaining battery fraction per UAV	Avoid energy exhaustion
9–32	24	x, y, z, vx, vy, vz per UAV	Know fleet position and motion
33–36	4	Herd centroid and x/y spread	Compact description of cow distribution
37–40	4	Current sensing intervals	Know the control already active
41–44	4	Current target speeds	Know live mobility targets
45–48	4	Current target altitudes	Know live altitude targets
49	1	Normalised mission time	Distinguish early and late decisions

Normalisation

Raw units are placed on comparable scales. Horizontal coordinates and velocities can be negative and use -1 to 1 . Most other values use 0 to 1 . Normalisation helps neural-network optimisation and the environment rejects values outside its declared bounds.

8. The 25 possible actions

ID	UAV role	Operation	Change
0	All	No operation	Keep current settings
1	Surveillance	Decrease sensing interval	-0.1 s
2	Surveillance	Increase sensing	$+0.1$ s

ID	UAV role	Operation	Change
		interval	
3	Surveillance	Decrease target speed	−1 m/s
4	Surveillance	Increase target speed	+1 m/s
5	Surveillance	Decrease target altitude	−5 m
6	Surveillance	Increase target altitude	+5 m
7	Patrol	Decrease sensing interval	−0.1 s
8	Patrol	Increase sensing interval	+0.1 s
9	Patrol	Decrease target speed	−1 m/s
10	Patrol	Increase target speed	+1 m/s
11	Patrol	Decrease target altitude	−5 m
12	Patrol	Increase target altitude	+5 m
13	Rapid-response	Decrease sensing interval	−0.1 s
14	Rapid-response	Increase sensing interval	+0.1 s
15	Rapid-response	Decrease target speed	−1 m/s
16	Rapid-response	Increase target speed	+1 m/s
17	Rapid-response	Decrease target altitude	−5 m
18	Rapid-response	Increase target altitude	+5 m
19	Strategic	Decrease sensing interval	−0.1 s
20	Strategic	Increase sensing interval	+0.1 s
21	Strategic	Decrease target speed	−1 m/s
22	Strategic	Increase target speed	+1 m/s

ID	UAV role	Operation	Change
23	Strategic	Decrease target altitude	−5 m
24	Strategic	Increase target altitude	+5 m

Every action is clipped to a safe role-specific bound. For example, repeated altitude-increase actions stop at the allowed upper altitude rather than increasing forever. Action 0 is important because sometimes changing nothing is best.

9. The reward: how the agent is told what ‘good’ means

The immediate window score rewards detection and throughput, penalises localisation error, delay, and loss, and then subtracts an energy term:

```
J = 3 × detection + 3 × throughput − 2 × RMSE − delay − loss
reward = J − energy_weight × (window_energy / window_energy_budget)
```

Every metric in J is normalised first. The weights 3, 3, 2, 1, and 1 express priorities. They are design decisions, not laws of nature, and must be justified or tested with sensitivity analysis.

9.1 A small made-up numerical example

Suppose the normalised values in one window are detection 0.8, throughput 0.5, RMSE 0.2, delay 0.1, and loss 0.0. Then $J = 3(0.8) + 3(0.5) - 2(0.2) - 0.1 - 0 = 3.4$. If the normalised energy penalty is 0.3, the reward is 3.1. This example teaches the calculation only; it is not a measured result.

Reward-design danger

A large positive reward does not automatically mean the physical system is good. Always report the original KPIs beside reward, inspect actions, and test whether the learned policy exploits a simulator shortcut.

10. Communication KPIs are measured per window

FlowMonitor counters increase throughout a run. If cumulative totals are read at 10, 20, and 30 seconds, the 30-second value includes earlier packets and is not the consequence of only the latest action. The implementation stores the previous counters and subtracts them. The difference belongs to the newest 10-second window.

KPI	Plain meaning	Window calculation
Throughput	Useful received data rate	New received bytes × 8 / window duration

KPI	Plain meaning	Window calculation
Delay	Average time for a received packet	New delay sum / new received packets
Loss	Percentage transmitted but not received	New lost packets / new transmitted packets
Detection	Fraction of sensing opportunities detected	Fusion receiver's newest window
RMSE	Typical localisation error	Root mean-square position error in newest window
Energy	Energy used by the fleet	Current cumulative energy – previous cumulative energy

11. The message handshake

The simulator and Python communicate using newline-separated JSON over a local TCP socket. TCP provides ordered delivery. IDs make the semantic order explicit. Python refuses a wrong episode ID, a skipped step, a malformed message, or an observation with a size other than 50.

```
{ "type": "transition", "episode_id": 4, "step_id": 0,
  "reward": 1.25, "terminated": false, "truncated": false,
  "observation": [50 numbers], "pdet": ..., "rmse_m": ... }

{ "episode_id": 4, "step_id": 0, "action": 9 }
```

The simulator blocks after sending a non-final state. It advances only after receiving the matching action. This guarantees that one action corresponds to one following window.

12. How Double DQN learns

12.1 The Q-network

The network takes 50 state numbers. They pass through 256 neurons, a ReLU activation, another 256 neurons and ReLU, and finally 25 outputs. Output 0 estimates the long-term value of action 0, output 1 the value of action 1, and so on. The greedy action is the output with the largest value.

12.2 The Bellman idea

A transition teaches the network that an action's value should be close to the reward received now plus the discounted value available next. The discount gamma is 0.99, meaning future reward matters strongly but is slightly discounted.

```
target = reward + gamma × (1 – done) × next_Q_value
```

12.3 Why Double DQN uses two networks

Ordinary DQN can overestimate values because one noisy network both picks and evaluates the largest next value. Double DQN uses the online network to choose the next action and the target network to value it. The target network is a delayed copy, updated every 1,000 environment steps.

12.4 Why replay memory exists

Consecutive simulator steps are strongly related. Training only on the newest transition can make learning unstable. Replay memory stores up to 100,000 transitions and samples mixed mini-batches of 128. Old and new experiences are therefore learned together.

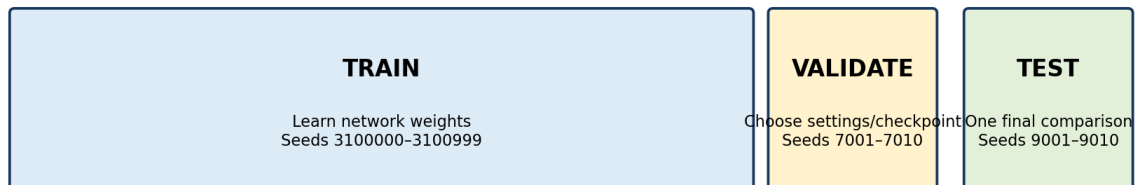
12.5 Exploration versus exploitation

At the beginning epsilon is 1.0, so actions are random. It decreases towards 0.05 over 30,000 steps. Random exploration discovers consequences; greedy exploitation uses the best current estimate. Evaluation always turns exploration off.

12.6 Loss, optimiser, and gradient clipping

Huber loss is less sensitive to a few very large errors than squared error. AdamW updates the weights with learning rate 0.0003. Gradient norm clipping at 10 prevents an unusually large batch from causing an extreme update.

13. Training, validation, and testing



Training data change the neural-network weights. Validation data choose hyperparameters or a checkpoint. Test data are used once for the final estimate. Looking at test results and then choosing a different model leaks test information and makes the reported performance optimistic.

Split	Can update weights?	Can choose model?	Can make final claim?
Training	Yes	Not by training return alone	No
Validation	No	Yes	No
Test	No	No	Yes, after the model is

Split	Can update weights?	Can choose model?	Can make final claim?
			frozen

The example full run with agent seed 31 automatically uses simulator seeds 3,100,000 through 3,100,999. Suggested validation seeds are 7001–7010 and test seeds 9001–9010. The exact sets should be declared before final evaluation.

14. What was changed in the code

File	Purpose/change
uav-isac-drl.cc	Added TCP bridge, episode/step handshake, live actions, 50-state construction, reward, termination, per-window KPI deltas, and immutable output arguments.
uav-isac-drl.cc	Fixed radar range to include UAV-to-ground altitude and logged actual live speed/altitude targets.
uav_isac_env.py	Added Gymnasium reset/step/close wrapper, process launch, socket validation, spaces, timeouts, command recording, and overwrite protection.
dqn_agent.py	Added neural networks, replay buffer, Double-DQN targets, epsilon policy, Huber optimisation, gradient clipping, target updates, save/load.
mock_uav_env.py	Added a tiny predictable environment for quick learning tests without ns-3.
tests/test_dqn.py	Verifies Bellman target, replay shapes, and learning of a known preferred action.
tests/test_protocol.py	Verifies message identity/action pairing and rejects the wrong state size.
tests/test_ns3_integration.py	Verifies live sensing, speed, altitude target, and physical altitude changes.
offline_rf_benchmark.py	Preserves and correctly names the released Random Forest method as a baseline.
generate_rf_dataset.py	Writes deterministic configurations and exact

File	Purpose/change
	commands for all newly generated RF runs.
select_checkpoint.py	Evaluates candidate checkpoints only on declared validation seeds.
evaluate_policies.py	Runs matched-seed baselines/DQN and writes raw results, method summaries, and paired differences.

15. What the tests proved

Test	Result	What it means	What it does not mean
Build	Passed	The C++ target compiles	The model is scientifically accurate
Bellman target	Passed	Double-DQN online-selection/target-evaluation formula works	Training will converge in ns-3
Replay buffer	Passed	Transitions and batch shapes work	Stored data are sufficient
Mock learning	Passed	The code learned known action 7	It learned good UAV control
Protocol tests	Passed	IDs and 50-state messages are enforced	TCP can never fail in deployment
ns-3 integration	Passed in 66.9 s	Live controls and physical altitude changed	The policy is better than baselines
Three-episode smoke	Completed	Training/save/validate/freeze/test pipeline runs	A DRL performance improvement

16. The measured smoke result—and how to read it

Method	Return	Detection	RMSE (m)	Throughput (Mbps)	Delay (ms)	Final-window energy (J)
Static	2.6033	0.5609	9.4829	0.0009184	11.5077	11239.26
Random	2.7903	0.5808	8.9565	0.0009600	11.3962	11239.32
Rule	2.7215	0.5665	9.1766	0.0009440	11.5781	11239.30
Double DQN	2.5076	0.5655	9.4867	0.0008800	11.9263	11239.20

This test used only seed 9002 and two action steps per policy. The DQN was trained for three 30-second episodes—six transitions total. Its epsilon was still approximately 1.0. All three validation checkpoints tied. The DQN return was lower than static in this seed. That is not a failure of the pipeline; it is the expected behaviour of an effectively untrained network.

Correct sentence

'The smoke experiment verified end-to-end execution but was not statistically powered to compare policy performance.'

Incorrect sentence

'Results prove that DRL significantly improves detection and energy efficiency.' The available data do not support this statement.

17. Commands: from setup to final comparison

17.1 Install Python packages and build

```
.venv/bin/python -m pip install -r scratch/uav-isac-drl/requirements.txt
./ns3 build uav-isac-drl -j 4
```

17.2 Run fast unit/protocol tests

```
.venv/bin/python -m unittest discover \
+ -s scratch/uav-isac-drl/tests -p 'test_*.py' -v
```

17.3 Run the slow live ns-3 integration test

```
RUN_NS3_INTEGRATION=1 .venv/bin/python -m unittest \
+ scratch/uav-isac-drl/tests/test_ns3_integration.py -v
```

17.4 Run a random-policy smoke episode

```
.venv/bin/python scratch/uav-isac-drl/random_policy_smoke.py \
+ --episodes 1 --seed 1001 --simulation-time 40 \
+ --output results/uav-isac-drl/random-smoke-new
```

17.5 Verify DQN quickly in the mock world

```
.venv/bin/python scratch/uav-isac-drl/train_dqn.py \
+ --environment mock --episodes 100 --learning-starts 64 --batch-size 32 \
+ --output results/uav-isac-drl/mock-training-new
```

17.6 Generate a new RF baseline dataset

```
.venv/bin/python scratch/uav-isac-drl/generate_rf_dataset.py \
+ --runs 300 --sampling-seed 20260818 --simulation-time 300 \
+ --output results/uav-isac-drl/rf-baseline-300
```

Use `--dry-run` first to inspect configurations and commands. The script refuses to append into an existing `run_summary.csv` because that could duplicate runs.

17.7 Fit the RF benchmark

```
.venv/bin/python scratch/uav-isac-drl/offline_rf_benchmark.py \
+ --csv results/uav-isac-drl/rf-baseline-300/run_summary.csv \
+ --output-dir results/uav-isac-drl/rf-model
```

17.8 Train one full DQN candidate

```
.venv/bin/python scratch/uav-isac-drl/train_dqn.py \
+ --environment ns3 --episodes 1000 --seed 31 --simulation-time 300 \
+ --learning-starts 2000 --batch-size 128 --epsilon-decay-steps 30000 \
+ --target-update-interval 1000 --checkpoint-every 25 \
+ --output results/uav-isac-drl/full/agent-seed-31
```

17.9 Select a checkpoint using validation seeds only

```
.venv/bin/python scratch/uav-isac-drl/select_checkpoint.py \
+ results/uav-isac-drl/full/agent-seed-31/checkpoint-episode-*.pt \
+ --validation-seeds 7001,7002,7003,7004,7005,7006,7007,7008,7009,7010 \
+ --simulation-time 300 \
+ --output results/uav-isac-drl/full/validation/selection.csv
```

17.10 Freeze and compare on unseen matched seeds

```
.venv/bin/python scratch/uav-isac-drl/evaluate_policies.py \
+ --methods static,rf-static,random,rule,dqn \
+ --checkpoint <path copied from selection.json> \
+ --rf-candidate results/uav-isac-drl/rf-model/offline_suggested_candidates.csv \
+ --seeds 9001,9002,9003,9004,9005,9006,9007,9008,9009,9010 \
+ --simulation-time 300 \
+ --output results/uav-isac-drl/final/comparison.csv
```

Runtime reality

The current debug ns-3 build is slow. The full 1,000-episode campaign can take many hours or days. Run a timed episode first, use a managed batch job, preserve logs, and do not substitute a tiny smoke run in the paper.

18. What the final paper must report

- Exact code revision, dependency versions, machine information, and complete commands.
- The full state, action, reward, termination, and truncation definitions.
- Training episodes, decisions, replay settings, learning-start threshold, epsilon schedule, and target-update interval.
- Separate and predeclared training, validation, and test seed sets.
- At least static, random, rule-based, released RF, and frozen-DQN baselines.
- Mean, standard deviation, 95% confidence interval, and paired per-seed differences for every KPI.
- Learning curves across independent agent seeds, not a single lucky curve.
- Action frequencies, action saturation/bound hits, and manual trajectory inspection.
- Ablations: no energy term, no communication state, no mobility actions, and possibly different control intervals.
- Limitations: simulator abstraction, partial observation, lack of field trials, and O-RAN deployment remaining future work.

18.1 Claims that are currently safe

- A genuine sequential Double-DQN environment has been implemented.
- The state/action handshake and Bellman/replay logic have automated tests.
- Sensing, speed, and altitude actions alter live simulation state.
- Communication KPIs are computed from window counter differences.
- The complete train-validate-freeze-test pipeline executes.

18.2 Claims that are not yet safe

- DRL significantly outperforms RF or rule-based control.
- DRL improves detection while reducing energy.
- The method generalises to farms or seeds not represented by the final test campaign.
- The policy is ready for deployment in an O-RAN xApp or on real UAVs.
- The original 300-plus thesis runs have been exactly reproduced.

19. Dataset recovery: what we know and what we do not

The public repository was audited at commit 8f5ce9977b37533e4fabffcc505751cb52998713. It contains one commit, no tags, and no CSV, pickle model, experiment script, or command manifest. The supplied thesis PDF has no embedded files. Searches of the available workspace and Downloads did not find the original run summaries or RF model.

Therefore the exact 300-plus dataset and commands cannot honestly be recovered. The new generator reconstructs a deterministic experiment from the released bounds and records all commands, but those new runs are not the missing original thesis runs. The released RF code uses 28 features; the thesis's count of 30 also included UE and gNB transmit power, which the released RF feature list did not use.

20. Troubleshooting

Symptom	Likely meaning	What to check
ns-3 exits before handshake	Build/runtime error	Open that episode's ns3.log and command.json
Connection timeout	Simulator did not open TCP port	Build target, port availability, ns3.log
Wrong observation size	C++ and Python state definitions differ	Count state insertions; keep exactly 50
Non-consecutive step	Protocol mismatch/stale message	Episode ID, step ID, only one controller
FileExistsError	A run path already contains evidence	Choose a new output directory; do not overwrite
No learning loss	Replay not ready	learning_starts, batch size, number of transitions
Epsilon stays near 1	Too few steps	Compare environment steps with epsilon_decay_steps
Policy repeats one action	Could be untrained, collapsed, or optimal	Validation returns, Q-values, action counts, bounds
Altitude changes but detection does not	Physics or time horizon issue	Confirm corrected slant range and enough follow-up windows
Result changes on rerun	Randomness/version difference	Seed, command.json, commit, dependencies, CPU/GPU

21. Frequently asked questions

Is Random Forest a form of DRL?

No. It is supervised learning here. It predicts a final score from a fixed configuration.

Why not use continuous actions?

DQN is designed for discrete actions. The 25-action set makes the first implementation testable. Continuous control could later use SAC or TD3, but that would be a new study.

Why does the agent see herd centroid rather than all cows?

Fifty observations keep the state compact and independent of herd size. The trade-off is loss of detailed spatial information.

Why is the first observation at 10 seconds?

A KPI window needs elapsed time and packets. Acting at time zero would use empty measurements.

Why separate terminated and truncated?

Battery exhaustion is a task terminal condition. Reaching the configured time limit is a truncation imposed by the experiment horizon.

Why does the static policy still use the RL bridge?

It sends action 0 through the same timing and measurement path, so comparisons differ by policy rather than instrumentation.

What does equal-seed mean?

Every method faces the same simulator RNG run. Paired differences then remove some scenario-to-scenario variation.

Can the smoke values go in the final Results section?

Only as labelled implementation verification. They cannot support a performance or significance claim.

Is the trained policy already an xApp?

No. The simulator retains an asynchronous xApp bridge, but safe deployment requires packaging, latency tests, policy constraints, monitoring, and fallback behaviour.

What is the next scientific step?

Run the predeclared full campaign, select only on validation seeds, freeze the model, and perform the matched multi-seed test once.

22. Final completion checklist

- [] Commit or archive the exact code revision used for experiments.
- [] Record dependency and ns-3/NR versions.
- [] Declare training, validation, test, and agent seeds before the final run.
- [] Generate the RF dataset and preserve configurations.json/commands.json.
- [] Train multiple DQN candidates and independent agent seeds.

- [] Select hyperparameters/checkpoint from validation results only.
- [] Freeze the selected model and checksum the checkpoint.
- [] Run every baseline on identical unseen seeds.
- [] Generate raw, summary, and paired-vs-static CSV files.
- [] Plot learning curves, KPI distributions, action use, and energy trade-offs.
- [] Replace the red result placeholder in the LaTeX paper.
- [] Remove all wording that calls RF 'offline DRL.'
- [] Have a second person reproduce at least one episode from command.json.
- [] Keep the limitations and distinguish simulation from field evidence.

Appendix A. Important output files

Output	Contains	Use
command.json	Exact process argument vector	Re-run or audit one episode
control_kpi_log.csv	One row per control window	Time series, action-effect inspection
run_summary.csv	Final whole-episode metrics and parameters	Episode-level comparison
ns3.log	Simulator output and live action traces	Debugging and manual verification
training.csv	Episode returns, epsilon, loss, steps, action counts	Learning diagnostics
checkpoint-episode-*.pt	Frozen candidate neural-network state	Validation selection
selection.csv/json	Validation returns and selected path	Proof that test seeds were not used
comparison.csv	One row per method and test seed	Raw final evidence
comparison-summary.csv	Means, SDs, 95% intervals	Paper summary table
comparison-paired-vs-static.csv	Matched-seed differences	Fair policy comparison

Appendix B. Source map

Implementation directory:

```
/home/awanthidel/workspace/MyNewWorkspace/ns-3-dev-nr-oran/scratch/uav-isac-drl
```

Verified post-fix smoke artifacts:

```
/home/awanthidel/workspace/MyNewWorkspace/ns-3-dev-nr-oran/results/uav-isac-drl/ns3-smoke-  
training-fixed  
/home/awanthidel/workspace/MyNewWorkspace/ns-3-dev-nr-oran/results/uav-isac-drl/ns3-smoke-  
validation-fixed  
/home/awanthidel/workspace/MyNewWorkspace/ns-3-dev-nr-oran/results/uav-isac-drl/ns3-smoke-  
comparison-fixed
```

The directories without the -fixed suffix are explicitly invalidated because they predate the altitude/radar-range correction.

Appendix C. Further reading

- R. Sutton and A. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.
- V. Mnih et al., 'Human-level control through deep reinforcement learning,' Nature, 2015. <https://doi.org/10.1038/nature14236>
- H. van Hasselt, A. Guez, and D. Silver, 'Deep Reinforcement Learning with Double Q-Learning,' AAAI, 2016. <https://doi.org/10.1609/aaai.v30i1.10295>
- M. Towers et al., 'Gymnasium: A Standard Interface for Reinforcement Learning Environments,' 2024. <https://arxiv.org/abs/2407.17032>

Revision record

Date	Version	Scope
18 August 2026	1.0	Initial beginner guide based on verified post-fix code and smoke artifacts

Document status

The explanation and implementation evidence are complete. Scientific performance results remain pending the full multi-seed campaign.